

Sistemas Distribuidos
(TA050)

Trabajo Práctico Tolerancia a Fallos

Money Laundering Analysis System

29 de junio de 2026

Nicolas Pablo Severini
Federico Penic
Marcos Garcia Neira

Índice

1. Introducción	2
2. Arquitectura General	2
3. Modelo de Procesamiento	2
4. Consultas Implementadas	2
5. Particionamiento y Escalabilidad	3
6. Coordinación mediante EOF	3
7. Tolerancia a Fallos	3
7.1. Healthcheckers	3
7.2. Persistencia de Estado	5
7.3. Write-Ahead Logging (WAL)	5
7.4. Checkpointing	5
7.5. Recuperación ante Fallos	6
7.6. Manejo de Duplicados	6
8. Conclusiones	7

1. Introducción

El objetivo de este trabajo práctico es diseñar e implementar un sistema distribuido para el análisis de operaciones financieras potencialmente asociadas al lavado de dinero.

El sistema procesa grandes volúmenes de transacciones bancarias y ejecuta múltiples consultas analíticas de forma concurrente, utilizando una arquitectura distribuida basada en microservicios comunicados mediante colas de mensajes. Cada componente del sistema realiza una etapa específica del procesamiento, permitiendo escalar horizontalmente y distribuir la carga de trabajo entre múltiples instancias.

La solución implementada sigue un modelo de procesamiento por etapas (pipeline), donde los datos son filtrados, transformados, agregados y finalmente almacenados para generar los resultados correspondientes a cada consulta.

A lo largo del desarrollo se priorizaron los principios de desacoplamiento, escalabilidad, paralelismo y tolerancia a fallos, incorporando mecanismos que permiten mantener la consistencia del procesamiento incluso ante caídas de nodos.

2. Arquitectura General

La arquitectura del sistema está compuesta por los siguientes elementos principales:

- **Gateway:** recibe las solicitudes de los clientes y coordina el ingreso de datos al sistema.
- **Workers:** realizan el procesamiento distribuido de las distintas consultas.
- **RabbitMQ:** actúa como middleware de mensajería permitiendo desacoplar productores y consumidores.
- **Cientes:** envían datasets de transacciones y reciben los resultados finales.
- **Sinks:** almacenan y entregan los resultados finales de cada consulta.

El sistema utiliza particionamiento por claves para distribuir el procesamiento entre múltiples instancias de un mismo nodo, garantizando paralelismo y balance de carga.

Los diagramas y flujo de datos pueden verse en el repositorio del proyecto:

<https://github.com/fedepenic/Tp-Integral-Sistemas-Distribuidos>

3. Modelo de Procesamiento

Las transacciones ingresan al sistema agrupadas en batches.

Cada batch contiene un conjunto de operaciones financieras asociadas a un cliente determinado. Los datos son distribuidos entre distintos nodos utilizando funciones de particionamiento que garantizan que todas las entidades relacionadas sean procesadas por la misma instancia cuando sea necesario.

El procesamiento se organiza como una secuencia de filtros, agregadores y operadores de combinación (*joins*), donde cada componente consume mensajes desde una cola de entrada y produce nuevos mensajes hacia las etapas posteriores.

Para indicar la finalización de los datos de un cliente se utilizan mensajes EOF (*End Of File*), que permiten coordinar correctamente las operaciones de agregación y cierre de resultados.

4. Consultas Implementadas

El sistema resuelve múltiples consultas de análisis financiero sobre el conjunto de transacciones. Entre las operaciones implementadas se incluyen:

- Filtrado de transacciones según moneda.

- Detección de cuentas con comportamiento anómalo.
- Cálculo de promedios y máximos por entidad bancaria.
- Identificación de cuentas con alta dispersión de transferencias.
- Combinación de resultados provenientes de múltiples flujos mediante operadores Join.

Cada consulta se implementa mediante una composición de nodos especializados que colaboran para producir el resultado final.

5. Particionamiento y Escalabilidad

La arquitectura fue diseñada para permitir la ejecución de múltiples instancias de cada componente.

Para ello se emplean funciones de particionamiento determinísticas que distribuyen los datos según claves derivadas de las cuentas bancarias o de las entidades involucradas.

Esta estrategia permite:

- Balancear la carga de procesamiento.
- Reducir la necesidad de comunicación entre nodos.
- Escalar horizontalmente agregando nuevas instancias.
- Mantener afinidad de datos para operaciones de agregación.

6. Coordinación mediante EOF

La coordinación entre componentes se realiza utilizando mensajes EOF.

Cada nodo mantiene un seguimiento de los EOF recibidos desde sus productores upstream. Una vez recibidos todos los EOF esperados para un cliente, el nodo puede considerar completado el procesamiento y emitir su correspondiente EOF downstream.

Este mecanismo permite coordinar correctamente operadores complejos como agregadores y joins distribuidos.

7. Tolerancia a Fallos

El sistema implementa múltiples mecanismos de tolerancia a fallos a lo largo de toda la pipeline de procesamiento, garantizando que ante caídas de nodos, reinicios o particiones de red, no se pierdan datos ni se generen resultados incorrectos.

7.1. Healthcheckers

El sistema cuenta con un servicio dedicado de monitoreo llamado `watcher` (`system/cmd/watcher/main.go`) que actúa como healthchecker externo de todos los nodos de la pipeline. Su arquitectura se compone de dos partes: un servidor de salud embebido en cada servicio y un proceso `watcher` replicado que realiza los chequeos.

Servidor de Salud Embebido

Cada nodo Go (gateway, cleaner, filtros, agregadores, joiners, sinks, contador, etc.) expone un puerto TCP de salud (puerto 9999 por defecto) mediante la función `health.StartIfEnabled()` (`internal/health/health.go`). Este servidor se activa únicamente cuando la variable de entorno `ENABLE_WATCHER=true`. La implementación es mínima: abre un `net.Listen` en el puerto configurado y, por cada conexión entrante, la acepta y la cierra inmediatamente sin intercambiar datos. Esto permite que el `watcher` verifique la vivacidad del nodo mediante un TCP ping sin necesidad de un protocolo de aplicación.

Servicio Watcher

El watcher es un contenedor Docker dedicado que periódicamente realiza conexiones TCP a cada servicio monitoreado. Se despliega con réplicas configurables (`N_WATCHERS`, por defecto 2) que se coordinan dividiendo el intervalo de chequeo en slots determinísticos alineados al reloj de pared, evitando que todos los watchers pinguen simultáneamente.

Configuración (variables de entorno):

- `WATCH_INTERVAL`: intervalo entre rondas de chequeo (por defecto 15s).
- `PING_TIMEOUT`: timeout por cada conexión TCP (por defecto 3s).
- `STARTUP_DELAY`: demora inicial antes de comenzar a monitorear (por defecto 30s), dando tiempo a que todos los servicios inicien.
- `SERVICES`: lista dinámica generada por `generate_compose.py` con todos los servicios y sus puertos de salud (ej. `cleaner_1:9999,gateway:9999,...`).

Flujo de operación:

1. El watcher itera sobre la lista de servicios obtenida de `SERVICES`.
2. Para cada servicio, intenta una conexión TCP (`net.DialTimeout`) con timeout de `PING_TIMEOUT`.
3. Si la conexión es exitosa, el servicio se marca como `alive`.
4. Si la conexión falla, el watcher consulta la API de Docker (`/var/run/docker.sock`) para localizar el contenedor por proyecto y nombre de servicio.
5. Si el contenedor finalizó con código 0 (`Exited (0)`), se considera `completed` y no se reinicia.
6. En caso contrario, se reinicia el contenedor mediante `POST /containers/{id}/restart`.

Coordinación entre réplicas: Cuando hay múltiples watchers, cada uno su slot mediante la función `sleepUntilNextSlot`:

$$\text{slot} = \frac{\text{interval}}{\text{watcherTotal}}, \quad \text{targetOffset} = (\text{watcherID} - 1) \times \text{slot}$$

Cada watcher espera hasta alcanzar su offset dentro del intervalo, distribuyendo uniformemente las rondas de chequeo.

Healthcheck Nativo de Docker (RabbitMQ)

Adicionalmente, RabbitMQ utiliza el healthcheck nativo de Docker:

healthcheck:

```
test: rabbitmq-diagnostics check_port_connectivity
interval: 5s
timeout: 3s
retries: 10
start_period: 50s
```

Todos los servicios declaran `depends_on: rabbitmq: condition: service_healthy`, asegurando que el broker de mensajes esté operativo antes de iniciar cualquier nodo de la pipeline.

Chaos Monkey (Pruebas de Resiliencia)

El sistema incluye un script `scripts/chaos.py` que mata servicios aleatoriamente para probar la capacidad de recuperación del watcher. Ejecutable mediante `make chaos`, selecciona servicios al azar (preservando al menos una réplica por tipo) y los termina con `SIGKILL`. El watcher detecta la caída en la siguiente ronda y reinicia los contenedores automáticamente.

7.2. Persistencia de Estado

Los nodos stateful (agregadores, joiners, contadores) persisten su estado en disco utilizando un `StateManager` (`internal/node/state_manager.go`). Cada instancia mantiene su propio directorio de estado en `/output/system/state/{service_instance}/`. El sistema soporta dos mecanismos de persistencia combinados:

- **Write-Ahead Log (WAL)**: registro línea-por-línea de cambios incrementales.
- **Checkpoint**: snapshot completo del estado, escrito atómicamente cada N batches.

Adicionalmente, los nodos escalables (filters con coordinación entre pares) persisten su estado de EOF (`internal/node/eofstate.go`) en archivos JSON en `/output/system/eof_state/{service_instance}.json`. Este archivo contiene:

- `seen_eofs`: EOFs ya recibidos.
- `eof_counts`: conteo de EOFs por cliente.
- `eof_forwarded`: EOFs ya reenviados downstream.
- `eof_completed`: clientes cuyo EOF se completó.

El Gateway utiliza su propio almacén de EOF dedicado (`cmd/gateway/eof_store.go`) en archivos de línea con EOF IDs, tanto para datos de clientes como para reportes. Cada ID de EOF se persiste mediante `append` atómico con `fsync` antes de ser considerado como procesado.

7.3. Write-Ahead Logging (WAL)

Cada nodo stateful mantiene un archivo `wal.log` con entradas JSON delimitadas por nueva línea. El flujo por cada batch es:

1. Procesar el batch → computar el delta (cambio de estado incremental).
2. `AppendWAL(batchID, delta)` → escribir JSON al WAL con `fsync` inmediato.
3. Aplicar el delta al estado en memoria.
4. Marcar el `batchID` como aplicado (dedup en memoria).
5. Opcionalmente, ejecutar checkpoint si se alcanzó el umbral de frecuencia.

Cada entrada del WAL (`WALEntry`) contiene tres campos:

- `batch_id`: identificador único del batch, usado para deduplicación.
- `seq`: número de secuencia monótono creciente.
- `delta`: payload opaco JSON con el cambio de estado específico de cada tipo de nodo.

La escritura del WAL sigue una secuencia atómica: archivo temporal → `fsync` → `rename`. Esto garantiza que la entrada está segura en disco antes de continuar.

7.4. Checkpointing

Periódicamente (cada 10.000 batches por defecto, configurable vía `CHECKPOINT_FREQ`), el `StateManager` escribe un checkpoint completo del estado (`checkpoint.json`) mediante el siguiente procedimiento atómico:

1. Serializar el estado completo más el número de secuencia actual del WAL (`CheckpointSeq`).
2. Escribir a archivo temporal `checkpoint.json.tmp`.

3. Ejecutar `fsync` sobre el archivo temporal.
4. Renombrar atómicamente `checkpoint.json.tmp` $\rightarrow$ `checkpoint.json`.
5. Rotar el WAL: cerrar el archivo actual y crear uno nuevo vacío.

La rotación del WAL posterior al checkpoint permite mantener el log acotado: las entradas anteriores al checkpoint ya están reflejadas en el snapshot y no necesitan ser replayadas durante una recuperación.

7.5. Recuperación ante Fallos

El proceso de recuperación (`StateManager.Recover()`) sigue estos pasos:

1. Cargar el checkpoint más reciente (`checkpoint.json`) que contiene el estado completo y el `CheckpointSeq`.
2. Cargar todas las entradas del WAL cuyo `Seq > CheckpointSeq`.
3. Reaplicar cada delta de las entradas del WAL al estado en memoria en orden de secuencia.
4. Poblar el mapa de deduplicación en memoria con los `batchID` de las entradas reaplicadas.
5. Reanudar el procesamiento normalmente.

El sistema maneja correctamente los siguientes escenarios de crash:

- **Crash entre AppendWAL y ApplyDelta:** el batch no está marcado como aplicado. Durante la recuperación, la entrada se reaplica (el `BatchDeduplicator` del WAL garantiza que no se duplique).
- **Crash después de ApplyDelta pero antes del ACK:** el delta ya está en estado y en el WAL, pero el dedup en memoria se perdió. Dado que los deltas son idempotentes (acumulaciones, máximos, sumas), la reaplicación produce el mismo estado.
- **Crash durante el checkpoint:** el archivo `checkpoint.json.tmp` queda huérfano; el checkpoint anterior intacto sigue siendo válido.

Además de la recuperación de estado, los nodos coordinados (Scalable) recuperan su estado de EOF desde el archivo JSON persistente, permitiendo saber qué EOFs ya fueron vistos, reenviados y completados antes del crash.

Los clientes también tienen su propio mecanismo de recuperación: cada cliente persiste su progreso en `/data/client_N/. {clientID}.progress`, permitiendo reanudar desde el último batch acknowledged tras un reinicio.

7.6. Manejo de Duplicados

El sistema implementa tres niveles de deduplicación:

BatchDeduplicator (en memoria)

Cada nodo stateful (agregador, joiner, contador) utiliza un `BatchDeduplicator` (`internal/dedup/dedup.go`) con un ring-buffer LRU de hasta 100.000 batch IDs (configurable vía `MAX_BATCH_IDS`). Funciona mediante:

- Un mapa `seen[batchID]` para verificación $O(1)$.
- Una lista ordenada FIFO para evicción: cuando se alcanza el límite, se elimina el 10 % más antiguo.
- Un wrapper `Wrap(fn, dedup)` que intercepta cada batch antes de la función de procesamiento: si el `batchID` ya fue visto, el batch se descarta silenciosamente.

Deduplicación de EOFs (Gateway)

El Gateway utiliza un `eofStore` con archivo de línea persistente. Cada EOF ID (ej. `client:1:eof`, `report:1:query:2:eof`) se escribe en disco con `fsync` antes de ser procesado. Si el mismo EOF llega nuevamente (por redelivery de RabbitMQ), el store lo detecta como duplicado y lo ignora.

Resiliencia de la red

El uso de RabbitMQ con acknowledgments (ACK/NACK) proporciona garantías de entrega *at-least-once*. Cuando un nodo se cae mientras procesa un mensaje, RabbitMQ redelivers el mensaje a otra instancia (o a la misma tras reconectarse). Los mecanismos de dedup mencionados aseguran que el procesamiento es *efectivamente once* (at-most-once en términos de efecto sobre el estado).

La combinación de WAL + checkpoint + deduplicación multi-nivel garantiza que el sistema puede tolerar fallos arbitrarios de cualquier nodo sin pérdida de datos ni corrupción de resultados, manteniendo la consistencia eventual del estado agregado.

8. Conclusiones

Durante el desarrollo de este trabajo se diseñó e implementó un sistema distribuido capaz de procesar grandes volúmenes de transacciones financieras mediante una arquitectura basada en comunicación asincrónica utilizando RabbitMQ.

A partir de la entrega inicial, el foco principal estuvo puesto en incorporar mecanismos de tolerancia a fallos que permitieran mantener la consistencia de los resultados aun frente a caídas arbitrarias de nodos. Para ello se implementaron mecanismos de persistencia incremental mediante Write-Ahead Logging (WAL), checkpoints periódicos del estado, recuperación automática tras reinicios y distintos niveles de deduplicación para evitar reprocesamientos indebidos.

Uno de los principales desafíos encontrados fue la correcta coordinación entre nodos distribuidos y el manejo de mensajes EOF, especialmente en componentes de agregación y operadores Join. La incorporación de persistencia del estado de coordinación permitió que los nodos pudieran recuperar correctamente su progreso luego de una falla y continuar el procesamiento sin perder información.

Las pruebas realizadas mediante inyección de fallos controlada demostraron que el sistema es capaz de recuperarse ante la caída de instancias individuales, manteniendo la disponibilidad del procesamiento y preservando la integridad de los resultados generados. Asimismo, la combinación de checkpoints y WAL permitió reducir significativamente los tiempos de recuperación respecto de una reconstrucción completa del estado.

Como trabajo futuro podrían explorarse mecanismos de replicación activa del estado, snapshots distribuidos y estrategias más avanzadas de consenso para eliminar puntos únicos de fallo y mejorar aún más las garantías de disponibilidad del sistema.

En conclusión, el sistema desarrollado cumple los objetivos planteados tanto en términos funcionales como de tolerancia a fallos, proporcionando una plataforma distribuida robusta para el análisis de operaciones financieras a gran escala.